

操作系统原理 作业-1 参考答案

第一章 Introduction（计算机系统概述）

一、选择题参考答案

1. 答案：B

操作系统属于系统软件（System Software），是最基本的系统软件，负责管理计算机硬件资源并为应用软件提供运行环境。

2. 答案：A

分时操作系统（Time-sharing OS）是多道程序设计的逻辑延伸，CPU 通过快速切换在多个作业之间分配时间，使得多个用户能以交互方式共享计算机。每个用户感觉整个计算机专属于自己。（教材 1.4 节）

3. 答案：D

操作系统作为系统软件，其核心功能包括：管理计算机系统的资源（作为资源分配器 resource allocator）和控制程序的执行（作为控制程序 control program）。（教材 1.1.2 节）

4. 答案：C

对称多处理（SMP）中，每个处理器都运行操作系统中的所有任务，各处理器之间是对等关系（peers），不存在主从关系。这与非对称多处理（Asymmetric Multiprocessing）不同，后者有一个 boss 处理器控制系统。（教材 1.3.2 节）

5. 答案：B

存储管理（Memory Management）负责跟踪内存使用情况、决定哪些进程和数据移入/移出内存、以及按需分配和回收内存空间。（教材 1.7 节）

6. 答案：B

中断向量（Interrupt Vector）是一个地址数组，存储在低端内存中，通过设备编号作为索引，提供各设备中断服务例程（Interrupt Service Routine）的入口地址。（教材 1.2.1 节）

7. 答案：C

引导程序（Bootstrap Program）存储在 ROM 或 EEPROM 中（统称为固件 firmware）。它在计算机上电或重启时执行，负责初始化系统各方面（从 CPU 寄存器到设备控制器到内存内容），并加载操作系统内核。（教材 1.2.1 节）

8. 答案：B

虚拟内存的主要优点是允许执行不完全在物理内存中的程序，将主存抽象为一个统一的大存储数组，实现逻辑内存（logical memory）与物理内存（physical memory）的分离，使程序员不必担心内存限制。（教材 1.4 节）

9. 答案：B

存储层次结构（Storage Hierarchy）中，从顶层（寄存器）到底层（磁带），存储容量递增、访问速度递减、每位存储成本递减。高层存储昂贵但速度快，低层存储便宜但速度慢。（教材 1.2.2 节，Figure 1.4）

10. 答案：D

用户程序不能直接修改模式位来进入内核模式，否则保护机制将被绕过。从用户模式切换到内核模式只能通过中断（interrupt）、陷阱（trap）或系统调用（system call）来触发，这些机制由硬件控制。（教材 1.5.1 节）

二、简答题参考答案

1. 操作系统的三个主要目的：

（1）方便性（Convenience）：为用户提供一个方便使用计算机的环境。操作系统屏蔽了底层硬件的复杂性，提供用户界面（如命令行、图形界面、触摸屏等），使用户无需了

解硬件细节即可使用计算机。

(2) 效率 (Efficiency)：以高效的方式管理计算机系统的硬件和软件资源。操作系统作为资源分配器 (Resource Allocator)，管理 CPU 时间、内存空间、文件存储空间、I/O 设备等资源，解决资源竞争，确保资源被公平、高效地使用。

(3) 可扩展性 (Ability to evolve)：允许在不影响已有服务的情况下，有效地开发、测试和引入新的系统功能。操作系统作为控制程序 (Control Program)，管理用户程序的执行，防止错误和对计算机的不当使用。

2. 内核模式与用户模式的区别及保护作用：

区别：系统通过硬件中的模式位 (mode bit) 区分两种模式。内核模式 (mode bit = 0) 下，操作系统代码执行，可以执行所有指令，包括特权指令；用户模式 (mode bit = 1) 下，用户程序执行，不能执行特权指令。

保护作用：可能造成危害的指令被设定为特权指令，只能在内核模式下执行。如果用户程序试图在用户模式下执行特权指令，硬件不会执行该指令，而是将其视为非法操作并产生陷阱 (trap) 给操作系统处理。

模式转换的触发条件：

- 用户模式 → 内核模式：通过中断 (interrupt)、陷阱/异常 (trap/exception) 或系统调用 (system call) 触发。系统调用本质上是通过 trap 指令实现的。
- 内核模式 → 用户模式：当操作系统完成服务后，将模式位设为 1，将控制权返回给用户程序。

(参见教材 Figure 1.10: 用户模式到内核模式的转换过程)

3. 多道程序设计与分时系统的区别和联系：

多道程序设计 (Multiprogramming)：

- 核心思想：在内存中同时保持多个作业 (jobs)，当一个作业需要等待 (如等待 I/O 完成) 时，CPU 切换到另一个作业执行，从而使 CPU 始终处于忙碌状态，提高 CPU 利用率。

- 目的：提高系统资源利用率，尤其是 CPU 利用率。
- 作业切换时机：当前作业等待 I/O 时才切换。

分时系统（Time Sharing / Multitasking）：

- 核心思想：是多道程序设计的逻辑延伸。CPU 通过极快地在多个作业之间切换来执行它们，切换频率很高，使每个用户都能在程序运行时与之交互。
- 目的：提供合理的响应时间（response time），支持用户交互。
- 作业切换时机：时间片（time slice）用完即切换，或等待 I/O 时切换。

联系：分时是多道程序设计的扩展。两者都在内存中保持多个程序，都使用 CPU 调度和多道程序设计技术。分时系统在多道程序设计的基础上增加了时间片轮转机制，以保证交互性。（教材 1.4 节）

4. DMA 的工作原理及优势：

工作原理：

- （1）设备驱动程序将 I/O 命令的参数（源地址、目标地址、传输字节数等）加载到 DMA 控制器的寄存器中。
- （2）DMA 控制器直接在 I/O 设备的缓冲区与内存之间传输整块数据，无需 CPU 干预。
- （3）数据传输完成后，DMA 控制器向 CPU 发送一个中断，通知传输完成。
- （4）整个数据块传输期间，CPU 可以执行其他任务。

相比中断驱动 I/O 的优势：

- 中断驱动 I/O：每传输一个字节（对于低速设备）就产生一个中断，CPU 需要频繁处理中断，开销很大。
 - DMA：每传输一个数据块才产生一个中断（而不是每个字节一个中断），大大减少了中断次数，释放 CPU 去处理其他任务，特别适合大量数据传输（如磁盘 I/O）的场景。
- （教材 1.2.3 节）

5. 缓存原理及缓存一致性问题：

缓存的基本原理：

信息通常存储在某个存储系统中（如主存）。当需要使用某个信息时，首先检查它是否

在缓存中。如果在缓存中（缓存命中），则直接从缓存中获取；如果不在（缓存未命中），则从原始存储中获取，同时将其副本放入缓存中，以备将来再次使用。缓存容量有限，因此缓存大小和替换策略的选择是重要的设计问题。

缓存一致性问题的产生：

在多处理器环境中，每个 CPU 都有自己的本地缓存（local cache）。同一个数据 A 的副本可能同时存在于多个 CPU 的缓存中。当一个 CPU 修改了其缓存中 A 的值时，必须确保这个更新立即反映到所有其他缓存中 A 的副本上，否则不同 CPU 读到的将是 A 的不同（过时的）值。

这个问题称为缓存一致性（Cache Coherency）问题，通常在硬件层面（低于操作系统层面）通过缓存一致性协议来解决。（教材 1.8.3 节）

三、分析题参考答案

1. 特权指令判断：

a. 设定定时器的值 —— 是特权指令。

理由：定时器用于确保操作系统维持对 CPU 的控制，防止用户程序无限期占用 CPU。如果用户可以修改定时器，就可以将其设为一个极大值甚至关闭定时器，从而独占 CPU 资源。

b. 读取时钟 —— 不是特权指令。

理由：读取时钟是一个只读操作，不会影响系统的正常运行。用户程序经常需要读取系统时间来记录时间戳等。

c. 清除内存 —— 是特权指令。

理由：如果允许用户程序任意清除内存，可能会破坏其他程序甚至操作系统本身的数据和代码，造成系统崩溃。

d. 产生陷阱指令 —— 不是特权指令。

理由：trap 指令是用户程序请求操作系统服务（系统调用）的标准方式。用户程序必须

能够执行 `trap` 指令，以便从用户模式切换到内核模式，请求操作系统服务。

e. 关闭中断 —— 是特权指令。

理由：中断是操作系统获取系统控制权的重要机制。如果用户程序可以关闭中断，操作系统将无法重新获得 CPU 控制权（如定时器中断被屏蔽），用户程序可以无限期运行。

f. 修改设备状态表中的条目 —— 是特权指令。

理由：设备状态表由操作系统维护，用于管理和跟踪 I/O 设备的状态。如果用户可以修改该表，可能导致设备管理混乱，影响其他进程的 I/O 操作。

g. 从用户模式切换到内核模式 —— 是特权指令。

理由：如果用户程序能直接切换到内核模式，就可以执行所有特权指令，整个保护机制将失效。只有通过中断、陷阱或系统调用才能间接触发模式切换。

h. 访问 I/O 设备 —— 是特权指令。

理由：I/O 操作需要通过操作系统统一管理，以防止用户程序直接操控设备造成冲突或安全问题。所有 I/O 指令都是特权指令。

2. 多种操作模式的分析：

（a）多种模式的两种可能用途：

用途一：虚拟机管理器（VMM）支持。

CPU 支持虚拟化时，通常有一个单独的模式用于指示 VMM 处于控制中。VMM 的权限高于普通用户进程（需要创建和管理虚拟机、改变 CPU 状态），但低于内核（不需要直接控制所有硬件）。三级模式：内核模式 > VMM 模式 > 用户模式。

用途二：内核组件的分级保护。

如 Intel x86 支持 Ring 0-3 四个特权级，可以将内核核心代码运行在 Ring 0，设备驱动运行在 Ring 1 或 Ring 2，用户程序运行在 Ring 3。这样即使设备驱动出现 bug，也不会直接影响内核核心的稳定性。

(b) 多种模式相比两种模式的优势：

多种模式提供了更细粒度的权限控制，实现了最小特权原则（Principle of Least Privilege）。

以 VMM 场景为例：在只有两种模式的系统中，VMM 只能运行在用户模式或内核模式。如果运行在用户模式，VMM 缺少管理虚拟机的必要权限；如果运行在内核模式，VMM 拥有了过多不必要的权限，可能危及系统安全。有了中间级别的模式，VMM 可以获得恰好足够的权限来管理虚拟机，而不会获得可能破坏宿主系统的权限。这提高了系统的安全性和稳定性。（教材 1.5.1 节，1.11.6 节）

3. 三种计算环境的对比分析：

(a) 客户端-服务器计算（Client-Server Computing）

系统架构：采用集中化架构。服务器（Server）提供服务，客户端（Client）请求服务，两者通过网络通信。服务器是系统的核心，所有客户端的请求都由服务器处理。

资源管理：资源集中在服务器端管理。服务器承担主要的计算和存储任务。操作系统需要有效管理大量并发客户端的请求，涉及进程管理、网络通信、安全认证等。

操作系统角色：服务器端 OS 需要支持高并发、高可用。分为计算服务器系统（compute-server）和文件服务器系统（file-server）两大类。

典型场景：Web 服务器向浏览器客户端提供网页内容；数据库服务器响应客户端的数据查询请求。

(b) 对等计算（Peer-to-Peer Computing）

系统架构：去中心化架构。所有节点地位平等，既可以是客户端也可以是服务器。不存在明确的主从关系，服务分散在网络中的各个节点上。

资源管理：资源分布在所有参与节点上，无中心化管理。每个节点自行管理本地资源，并通过发现协议（discovery protocol）或集中查找服务来定位其他节点提供的服务。

操作系统角色：OS 需要支持节点同时扮演客户端和服务器的双重角色，需要提供网络发现、资源共享和分布式通信机制。

典型场景：BitTorrent 文件共享——每个用户在下载文件的同时也向其他用户上传已下载的部分；Skype 使用混合 P2P 方式实现语音/视频通话。

（c）云计算（Cloud Computing）

系统架构：基于虚拟化技术，通过网络提供计算、存储和应用服务。是虚拟化的逻辑延伸，用户按需使用，按用量付费。包括公有云、私有云和混合云。

资源管理：资源通过虚拟化技术实现池化和弹性分配。VMM 管理多个虚拟机，云管理工具（如 VMware vCloud、Eucalyptus）在更高层面管理资源。

提供三种服务模型：SaaS（软件即服务）、PaaS（平台即服务）、IaaS（基础设施即服务）。

操作系统角色：多层 OS 共存——虚拟机内运行传统 OS，VMM 管理虚拟机，云管理平台可视为新型 OS 负责资源编排和调度。

典型场景：Amazon EC2 提供 IaaS——用户租用虚拟机运行自己的应用，按需扩展计算资源。（教材 1.11.4-1.11.7 节）